CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Technology and Engineering — CSPIT-IT

ITUE301 — Advanced Web Development Frameworks

B.Tech. – CSE, CE, IT | Semester: 5th | AY 2026–27

OPEN-BOOK PRACTICAL EXAMINATION

SET A — Hospital Appointment System

Duration	2 Hours
Total Marks	20
Tech Stack	React + Express.js + MongoDB (Mongoose)
Examination Type	Open-Book Practical Examination
Syllabus Coverage	Practical 1 to Practical 5
Internet	Permitted for documentation/reference
AI Tools	Permitted
Code Sharing	Strictly prohibited

General Instructions

1. Read the complete question before starting implementation.
2. Attempt all five tasks. Partial implementation will receive partial marks.
3. Use:
   - React for frontend tasks
   - Express.js for backend tasks
   - MongoDB with Mongoose for database tasks
4. Your GitHub repository must be public and named:

```
itue301-exam-[your-roll-number]-[batch]
```

Example: itue301-exam-24cse001-A

5. Use .env for the MongoDB connection string. Do not hardcode the connection string.
6. Commit an .env.example file. Do not commit .env.
7. The backend must start using:

```
node server.js   (or)   npm start
```

8. Internet access is permitted for reading documentation and references.
9. AI tools such as ChatGPT, GitHub Copilot, Claude and Gemini are permitted. Students must be able to explain the submitted code during viva.

10. Sharing code with another student is strictly prohibited.
11. Submit the GitHub repository link along with SHA (commit ID) before the examination ends.
12. Include a README.md containing basic instructions for running the frontend and backend.

Scenario: Hospital Appointment System

MedCare Plus is a private hospital that wants to maintain basic information about doctors, patients and appointments.

You are required to develop selected parts of a Hospital Appointment System.

Data Entities

Use the following data structures.

Patient

- name
- email
- phone
- bloodGroup
- age

Doctor

- name
- email
- specialisation
- available

Appointment

- patientId
- doctorId
- date
- timeSlot
- status
- reason

Appointment status must be one of:

```
pending   confirmed   cancelled
```

Task 1 — React Component Architecture

4 Marks

Create the basic React frontend for the Hospital Appointment System.

Create the following page/components:

- HomePage
- DoctorsPage
- BookingPage
- AppointmentCard

Place reusable components inside a suitable /components folder.

The AppointmentCard component must accept the following props:

```
patientName doctorName date timeSlot status
```

Display all five values.

The appearance of the appointment status should change according to its value. For example:

- confirmed
- pending
- cancelled

may use different CSS classes.

Use props to pass appointment data from the parent component to AppointmentCard.

Detailed UI styling is not required. Focus on component structure, composition and props.

Task 2 — React Routing and State Management

4 Marks

Configure React Router with the following routes:

Route	Component
/	HomePage
/doctors	DoctorsPage
/booking	BookingPage

Create a navigation component containing links to all three routes.

Navigation must be performed using React Router links without a full-page reload.

In BookingPage, create a simple appointment form containing at least:

- Patient name
- Doctor name
- Date
- Time slot

Use useState to manage the form data.

At least two state values must be used meaningfully. For example:

```
form data selected doctor
```

Display the entered patient name or another selected value on the page as the state changes.

Task 3 — Express REST API + Middleware

4 Marks

Create an Express backend for managing appointments.

Implement the following three REST endpoints:

Method	Endpoint	Purpose
GET	/api/v1/appointments	Return all appointments
POST	/api/v1/appointments	Create a new appointment
GET	/api/v1/doctors	Return all doctors

For this task, appointments and doctors may initially be stored in in-memory arrays (e.g., a small hardcoded doctors list on the server). MongoDB implementation is assessed separately in Task 5.

Create a custom requestLogger middleware.

For every request, it should log:

```
[METHOD] [PATH] [TIMESTAMP]
```

Example:

```
[GET] /api/v1/appointments [2026-08-20T10:15:20.000Z]
```

Apply the middleware globally.

Also implement a global error-handling middleware as the last middleware in the application.

It should return a structured JSON response instead of exposing the raw error stack.

Use appropriate HTTP status codes. In particular:

- 200 for successful GET
- 201 for successful POST
- 500 for an unhandled server error

Test the APIs using Postman or Thunder Client.

Task 4 — REST API Consumption in React

4 Marks

Use the Express API developed in **Task 3** from the React frontend.

In DoctorsPage, retrieve doctor information using:

```
GET /api/v1/doctors
```

Use fetch or Axios.

Use useEffect() so that the API request is made when the component is mounted.

Maintain three states:

```
data, loading and error
```

The page must:

1. Display a loading message/indicator while the request is in progress.
2. Display an error message if the request fails.
3. Display the doctor data after a successful request.
4. Display at least:
    - Doctor name
    - Specialisation
    - Availability

The doctor information must be rendered from the API response and **must not be hardcoded in the React component**.

The API call should follow the asynchronous pattern.

Task 4 consumes the Express API from Task 3. **MongoDB is not involved in this flow.**

Task 5 — MongoDB + Mongoose Schema Design and Validation

4 Marks

Create a **separate Mongoose-based implementation** for the hospital database. Connect to MongoDB using Mongoose.

Create the following three schemas.

Patient Schema

Field	Requirement
name	String, required
email	String, required, unique
phone	String
bloodGroup	String, enum
age	Number

The allowed blood groups are:

```
A+   A-   B+   B-   AB+   AB-   O+   O-
```

Doctor Schema

Field	Requirement
name	String, required
email	String
specialisation	String, required
available	Boolean, default true

Appointment Schema

Field	Requirement
patientId	Reference to Patient
doctorId	Reference to Doctor
date	Required
timeSlot	Required
status	Enum
reason	Maximum 300 characters

The status field must use:

```
pending   confirmed   cancelled
```

with pending as the default value.

Use Mongoose references for:

```
patientId → Patient doctorId → Doctor
```

Connect MongoDB using a connection string stored in .env. For example:

```
MONGO_URI=your_mongodb_connection_string
```

Create at least one MongoDB operation that demonstrates that the schema is working.

Demonstrate at least one validation failure, such as:

- missing required field
- invalid blood group
- invalid appointment status
- reason exceeding 300 characters

Return a meaningful error response instead of exposing the raw Mongoose error object.

Submission Requirements

The GitHub repository must contain:

```
itue301-exam-[roll-number]-[batch]/
│
├── frontend/
│   ├── src/
│   └── package.json
│
├── backend/
│   ├── models/
│   ├── server.js
│   └── package.json
│
├── .env.example
├── .gitignore
└── README.md
```

The README must briefly explain:

1. Project name
2. Frontend setup and run command
3. Backend setup and run command
4. MongoDB setup
5. Required environment variables

Report / Evidence

Submit a short PDF report containing three screenshots:

Screenshot 1 — React Application

Show the Hospital Appointment System running in the browser.

Screenshot 2 — REST API

Show Postman/Thunder Client displaying a successful request to one of the Express endpoints.

Screenshot 3 — MongoDB

Show MongoDB Compass or MongoDB Atlas displaying the created database/document.

PDF filename:

```
[RollNo]_SetA_Report.pdf
```